

PRACTICAL -WEEK 6

Task - 2:

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int signal)
{
    if (signal == SIGINT)
        printf("\nSIGINT received.\n");

    else if (signal == SIGTERM)
        printf("SIGTERM received.\n");

    else if (signal == SIGUSR1)
        printf("SIGUSR1 received.\n");
}

int main()
{
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Program is running.\n");
    printf("PID: %d\n", getpid());

    while (1)
    {
        printf("Waiting for signals...\n");
        sleep(5);
    }

    return 0;
}
```

OUTPUT:

[illegible]

OBSERVATION:

The program successfully captured and handled the SIGINT, SIGTERM, and SIGUSR1 signals using POSIX signal handling. A signal handler was registered for each signal, and when a signal was sent to the running process, the corresponding handler was executed and the appropriate message was displayed. The experiment demonstrated asynchronous event handling, where signals can be received by a process at any time during its execution. Thus, the role of signal handlers in responding to and managing asynchronous events was successfully demonstrated.